Оптимизация задачи о целочисленном рюкзаке и (min, +)-свертке при помощи использования алгоритмов быстрого произведения многочленов

Украинский Александр 3821Б1МА1

Декабрь 2024

Аннотация

В данной работе рассматривается оптимизация алгоритма поиска оптимального решения задачи о неограниченном целочисленном рюкзаке на основе статьи Ю. Нестерова [1]Fast Fourier Transform and its Applications to Integer Knapsack Problems. В статье предлагается алгоритм поиска оптимума при использовании алгоритма быстрого дискретного преобразования Фурье. Итоговая сложность полученного алгоритма O($\|a\|_1 ln(\|a\|_1) ln(n) + b ln^2(n)$), где b - объём рюкзака, n - число предметов, $\|a\|_1$ - норма вектора весов предметов. Однако битовая сложность алгоритма не является полиномиальной, так как при оценке не учитывается рост сложности в зависимости от объемов входных данных. В качестве глобальной задачи работы ставится разработка эффективного алгоритма для поиска оптимального решения задачи о неограниченном целочисленном рюкзаке. В качестве решения проблемы предлагается использовать представленный в статье алгоритм с учетом того, что коэффициенты полинома, кодирующего рюкзак, представлены в виде логарифмов самих коэффициентов. Подобное представление позволяет избежать экспоненциального роста сложности. В статье проверяется гипотеза о возможности построения рандомизированного алгоритма со сложностью O($\|a\|_1 ln(\|a\|_1) ln(n)$) на основе случайного выбора первообразного корня в матрице Вандермонда. В работе рассматриваем одну из наиболее распространенных реализаций алгоритма БПФ - Кули-Тьюки. Также в работе рассматривается метод двойного экспоненциального представления с целью эффективного приближения при восстановлении ответа и общей задачи о (min, +)-свертке. Предприняты поппытки улучшить метод двойного эфкспоненциального представления.

Содержание

- Вычисление оптимума задачи о рюкзаке с целочисленными коэффициентами при помощи БПФ
- Рассмотрение частного случая
- Идея реализации алгоритма
- Программная реализация алгоритма
- Выводы
- Метод двойного экспоненциального представления
- Программная реализация двойного экспоненциального представления
- Заключение
- Ссылки и использованная литература

1 Вычисление оптимума задачи о рюкзаке с целочисленными коэффициентами при помощи БПФ

Вычисление коэффициентов полинома рюкзака

Из [1] также будем рассматривать задачу о рюкзаке с целочисленными коэффициентами. Пусть $a = \{a^{(1)}, a^{(2)}, ..., a^{(n)}\}$ - набор коэффициентов. $a \in \mathbb{Z}_+^n$. Рассмотрим полином ограниченного рюкзака

$$B_a^u(b) = \{x \in \prod_{i=1}^{n} \{0, 1, ..., u^{(i)}\} : \langle a, x \rangle = b\}$$

.

Рассмотрим также функцию $f_{B_a^u(b)}(t)$, которую назовем порождающей. Несложно убедиться, что

$$f_{B_a^u(b)}(t) \equiv \sum_{b=0}^{\infty} \mathcal{N}(B_a^u(b)) \cdot t^b$$

По лемме из [1] известно, что

$$f_{B_a^u(b)}(t) = \prod_{i=1}^{n} (\sum_{k=0}^{u^i} t^{ka^{(i)}}).$$

Рассмотрим функцию

$$f(t) = \prod_{i=1}^{n} p_i(t),$$

которая представлена в виде произведения полиномов. По построению алгоритма БПФ имеем, что коэффициенты $f(t)$ могут быть вычислены за $O(Dln(D)ln(n))$ арифметических операции, где $D = \sum_{i=1}^{n} u^i$.

Отсюда получаем, что вычисление всех $\langle a, u \rangle$ коэффициентов функции $f_{B_a^u(b)}(t)$ может быть вычислено БПФ за

$$O(\langle a, u \rangle ln(\langle a, u \rangle) ln(n))$$

арифметических операций.

Вычисление оптимального значения задачи о рюкзаке

Рассмотрим вектор $c \in \mathbb{R}^n$. Для $\mathcal{R} \subset \mathbb{R}^n$ определяем порождающую функцию

$$g_{\mathcal{R}}(c) = e^{\langle c,x \rangle}.$$

Введем на множестве $\mathcal{R}$ характеристическую функцию

$$\psi_{\mathcal{R}}(c) = ln(g_{\mathcal{R}}(c)).$$

Обозначим $\xi_{\mathcal{R}}(c) \equiv max(\langle c, x \rangle)$.
Справедливо соотношение

$$\xi_{\mathcal{R}}(c) \leq \psi_{\mathcal{R}}(c) \leq \xi_{\mathcal{R}}(c) + ln\mathcal{N}(\mathcal{R}) \implies \xi_{\mathcal{R}}(c) \leq \mu\psi_{\mathcal{R}}(c/\mu) \leq \xi_{\mathcal{R}}(c) + \mu ln\mathcal{N}(\mathcal{R})$$

, $\mu > 0$.
Определим дополненную порождающую функцию

$$F_{B_a^u}(c,t) = \sum_{g=0}^{\infty} g_{B_a^u(b)}(c) \cdot t^b \equiv \sum_{b=0}^{\infty} exp(\psi_{B_a^u(b)}(c)) \cdot t^b$$

По теореме из [1]

$$F_{B_a^u}(c,t) = \prod_{i=1}^{n} (\sum_{k=0}^{u^{(i)}} e^{kc^{(}i)} t^{ka^{(i)}}).$$

Найдем $f^* = max_{x \in \mathbb{Z}_n^+} \{\langle c, x \rangle : \langle a, x \rangle = b\} \equiv \xi_{B_a^\infty(b)}(c)$
Мы ищем целочисленное решение. Положив $\mu < \frac{1}{n} ln(1+b)$ имеем

$$-1 + \mu\psi_{B_a^\infty(b)}(c/\mu) < f^* \leq \mu\psi_{B_a^\infty(b)}(c/\mu) \qquad (1).$$

Таким образом, по теореме из [1] получаем, что общая сложность алгоритма составляет

$$O(\|a\|_1 ln(\|a\|_1) ln(n) + b \cdot ln^2 n)$$

арифметических операций.

2 Рассмотрение частного случая

Проверим на конкретном примере, что выражение (1) дает верное значение оптимума f^*.
Положим $n = 2, c = (5,7)^T, a = (1,2)^T, b = 10$.
$f^* = \xi_{B_a^\infty(b)}(c) = \xi_{B_a^\infty(b)}((5,7)^T)$.
Перебором несложно убедиться, что значение $f^* = 50$.
$\mathcal{N}(B_a^\infty(10)) = 6$.
$\mu = \frac{ln(1+10)}{2} = \frac{ln11}{2}$.
Имеем $x \in \{(0,5),(2,4),(4,3),(6,2),(8,1),(10,0)\}$.
$g_{B_a^\infty(10)}((10ln11, 14ln11)^T) = e^{70ln11} + e^{76ln11} + e^{82ln11} + e^{88ln11} + e^{94ln11} + e^{100ln11} \approx 50.000006$.
Отсюда $49.000006 < f^* \leq 50.000006 \implies f^* = 50$, что сответствует значению оптимума, полученному при переборе.

3 Идея реализации алгоритма

Рассмотрим битовую сложность полученного алгоритма. Мы рассматриваем вычисление оптимума задачи при помощи характеристической функции $g_{\mathcal{R}}(c) = e^{\langle c,x \rangle}$, которая напрямую зависит от порождающей функции $g_{\mathcal{R}}(c) = e^{\langle c,x \rangle}$. Таким образом, алгоритм будет зависеть не только от нормы вектора a, длины полинома, кодирующего рюкзак n и вектора ограничения b, но еще и от вектора c. Таким образом, битовая сложность алгоритма составляет

$$O((\|a\|_1 ln(\|a\|_1) ln(n) + b \cdot ln^2 n) \cdot n \cdot c_{max}),$$

где c_{max} - максимальный элемент вектора c.
Значение c_{max} может быть достаточно большим. В качестве идеи реализации алгоритма, сложность которого от c не зависит, предлагается на вход подавать не сами коэффициенты полинома, а логарифмы коэффициентов.
Теперь любой коэффициент $p_i \equiv e^a$ представляем в виде a. Тогда задача сводится к поиску эффективной аппроксимации $e^c = e^a + e^b$. В статье Э. Б. Саффа [2] Geometric Convergence to e^{-z} by Rational Functions with Real Poles утверждается, что существует эффективный метод приближения функции e^{-z} функцией вида

$$R_n(z) = \frac{P_{n-1}(z)}{(1 + \frac{z}{n})^n}$$

, где $deg(P_{n-1}) \leq n - 1$. Утверждение справедливо на ограниченном множестве

$$S(\theta) := \{z \in \mathbb{C} : |arg(z)| < \theta\}.$$

при $0 \leq \theta < \pi$. Причем

$$\overline{\lim_{n\to\infty}} \|e^{-z} - \mathbb{R}_n(z)\|_{S(\theta)}^{1/n} \leq \frac{1}{q}\Big(\frac{1 + sin(\theta/2)}{1 - sin(\theta/2)}\Big) < 1$$

Естественно полагать, что данное приближение имеет погрешность. Для линейного роста ошибки необходимо выполнение условия того, чтобы между двумя комплексными векторами угол не превышал какой-либо константы, не превышающей π. Возникает вопрос, можно ли изменяя первообразный корень матрицы Вандермонда эффективно вычислить все коэффициенты итогового полинома.
Для проверки гипотезы составим выборку полиномов со случайными коэффициентами. Будем фиксировать угол между числами на каждом шаге алгоритма БПФ, представленными в виде комплексных векторов. В том случае, когда аппроксимация невозможна без потери точности будем сохранять результат в виде суммы до следующего шага рекурсии. Будем случайно выбирать первообразный корень матрицы Вандермонда. На основе выборки сделаем вывод о практической применимости данного метода.

4 Программная реализация

Для проверки гипотезы создадим случайную выборку. Рассмотрим задачу произведения двух многочленов задаваемой степени с коэффициентами из некоторого диапазона. Из постановки задачи коэффициенты также являются случайными числами из $\mathbb{Z}_+$ в некотором диапазоне. Реализуем прямой и обратный алгоритмы Кули-Тьюки и на их основе реализуем алгоритм произведения двух полиномов. Генерируя случайные коэффициенты массивов фиксируем среднее число шагов для достижения заданной точности, средний прирост покрытых коэффициентов и число шагов, за которое в среднем покрывается половина коэффициентов. В дальнейшем будем называть покрытыми коэффициенты, которые хотя бы раз при запуске алгоритма были представлены в виде одного числа.
Псевдокод:

```
input: n
complex a[n]
double first_step[k]
double increase[k]
int half[k]
int ans[p]
for(p)
    b[n]
    for(k)
    summa = 0
        j = random(a[n])
        ...
        b = fpp(b, j)
        if (summa / n >= 0.5)
            half.append(i)
        if (summa / n >= 0.99)
            ans.append(i)
print(E(ans))
print(E(increase))
print(E(half))
print(E(first_step))
```

Исследование проводим на случайных данных. Коэффициенты полинома - случайные числа, находящиеся в диапазоне от 0 до 25 за исключением коэффициентов, кодирующих старшую степень. Они находятся в диапазоне от 1 до 25. Длина полинома $N = 2^n$, следовательно можно выбрать $\frac{n}{2}$ первообразных корня из 1. Первообразный корень выбирается случайно.

Исследуем покрытие 99% коэффициентов полинома при помощи алгоритма быстрого произведения. Будем фиксировать число шагов, за которое было покрыто 99% коэффициентов, $\frac{1}{2}$ коэффициентов, процент покрытых коэффициентов на первом шаге и средний прирост коффициентов за n шагов.

Программная реализация:

```
import cmath
import math
import numpy as np
```

```python
import random
f1 = open("C:\\Users\\eugen\\PycharmProjects\\PROJECTFOLDER\\ikp.txt", "w")


def p_root(u, v):  # возвращает первообразный корень
    return cmath.exp(2 * cmath.pi * v * 1j / u)


def mk_p(u):  # возвращает список индексов первообразных корней
    pr_l = []
    for pri in range(1, u, 2):
        pr_l.append(pri)
    return pr_l


def deg(x, y):  # возвращает степень результирующего полинома
    st = 1
    while st < max(x, y):
        st <<= 1
    st <<= 1
    return st


def ang_comp(x, y):  # возвращает угол между комплексными векторами
    if x.real == 0 and x.imag == 0 or y.real == 0 and y.imag == 0:
        return 0
    if np.sign(x.real) * np.sign(y.real) >= 0
        and np.sign(x.imag) * np.sign(y.imag) >= 0:
            return math.fabs(math.fabs(np.angle(x))
                - math.fabs(np.angle(y)))
    if np.sign(x.real) * np.sign(y.real) <= 0
        and np.sign(x.imag) * np.sign(y.imag) <= 0:
            return math.pi - ang_comp(x, -y)
    if np.sign(x.real) * np.sign(y.real) <= 0
        and np.sign(x.imag) * np.sign(y.imag) >= 0:
            return math.fabs(np.angle(x) - np.angle(y))
    if np.sign(x.real) * np.sign(y.real) >= 0
        and np.sign(x.imag) * np.sign(y.imag) <= 0:
            if x.real >= 0 and y.real >= 0:
                return math.fabs(np.angle(x))
                    + math.fabs(np.angle(y))
            if x.real <= 0 and y.real <= 0:
                return 2 * math.pi - math.fabs(np.angle(x))
                    - math.fabs(np.angle(y))


def app(x, even, odd, w, wn, u):  # сложение элементов на шаге бпф
    for elem in range(0, int(len(x)/2)):
        # тестовый вариант
        if len(even[elem]) == 1 and len(odd[elem]) == 1:
```

```
        if ang_comp(even[elem][0], w * odd[elem][0])
        < 2 * math.pi / 3:
            x[elem] = [even[elem][0] / (u + 1)
                + w * odd[elem][0] / (u + 1)]
            x[int(len(x) / 2) + elem] = [even[elem][0] / (u + 1)
                - w * odd[elem][0] / (u + 1)]
        else:
            x[elem] = [even[elem][0] / (u + 1),
                w * odd[elem][0] / (u + 1)]
            x[int(len(x) / 2) + elem] = [even[elem][0] / (u + 1),
                -w * odd[elem][0] / (u + 1)]
            x[elem].sort(key=lambda ang: np.angle(ang))
            x[int(len(x) / 2) + elem].sort(key=lambda ang:
            np.angle(ang))
    else:
        x[elem] = []
        x[int(len(x) / 2) + elem] = []
        for elem1 in range(0, len(even[elem])):
            x[elem].append(even[elem][elem1] / (u + 1))
            x[int(len(x) / 2) + elem].append(even[elem][elem1]
             / (u + 1))
        for elem1 in range(0, len(odd[elem])):
            x[elem].append(w * odd[elem][elem1] / (u + 1))
            x[int(len(x) / 2) + elem].append(-w * odd[elem][elem1]
             / (u + 1))
        x[elem].sort(key=lambda ang: np.angle(ang))
        x[int(len(x) / 2) + elem].sort(key=lambda ang:
        np.angle(ang))
    if len(x[elem]) >= 3:
        # iteration = 0
        while True:
            if len(x[elem]) == 2:
                if ang_comp(x[elem][0], x[elem][1])
                < 2 * math.pi / 3:
                    x[elem][0] += x[elem][1]
                    x[elem].remove(x[elem][1])
                break
            cit = 0
            c_ang = 0
            for iteration in range(0, len(x[elem])):
                itl = iteration % len(x[elem])
                itl1 = (iteration + 1) % len(x[elem])
                if c_ang < ang_comp(x[elem][itl], x[elem][itl1])
                < 2 * math.pi / 3:
                    c_ang = ang_comp(x[elem][itl], x[elem][itl1])
                    cit = iteration
            x[elem][cit] += x[elem][(cit + 1) % len(x[elem])]
            x[elem].remove(x[elem][(cit + 1) % len(x[elem])])
            x[elem].sort(key=lambda ang: np.angle(ang))
    elem2 = elem + int(len(x) / 2)
```

```
        if len(x[elem2]) >= 3:
            while True:
                if len(x[elem2]) == 2:
                    if ang_comp(x[elem2][0], x[elem2][1])
                    < 2 * math.pi / 3:
                        x[elem2][0] += x[elem2][1]
                        x[elem2].remove(x[elem2][1])
                    break
                cit = 0
                c_ang = 0
                for iteration in range(0, len(x[elem2])):
                    itl = iteration % len(x[elem2])
                    itl1 = (iteration + 1) % len(x[elem2])
                    if c_ang < ang_comp(x[elem2][itl], x[elem2][itl1])
                    < 2 * math.pi / 3:
                        c_ang = ang_comp(x[elem2][itl], x[elem2][itl1])
                        cit = iteration
                x[elem2][cit] += x[elem2][(cit + 1) % len(x[elem2])]
                x[elem2].remove(x[elem2][(cit + 1) % len(x[elem2])])
                x[elem2].sort(key=lambda ang: np.angle(ang))
        w *= wn
    return x


count1 = 0  # углов на ФФТ
count2 = 0  # углов на ИФФТ
tlen = 0  # итоговая длина массива коэффициентов


def fft(x, u):
    global count1
    global tlen
    global count2
    if len(x) == 1:
        return
    even = []
    odd = []
    for elem in range(0, len(x)):
        if elem % 2:
            odd.append(x[elem])
        else:
            even.append(x[elem])
    fft(even, u)
    fft(odd, u)
    w = complex(1)
    wn = p_root(int(len(x)), u)
    p = app(x, even, odd, w, wn, 0)
    x = p
    return x
```

```python
def i_fft(x, u):
    global count1
    global tlen
    global count2
    if len(x) == 1:
        return
    even = []
    odd = []
    for elem in range(0, len(x)):
        if elem % 2:
            odd.append(x[elem])
        else:
            even.append(x[elem])
    i_fft(even, u)
    i_fft(odd, u)
    w = complex(1)
    wn = p_root(int(len(x)), (-1) * u)
    p = app(x, even, odd, w, wn, 1)
    x = p
    return x


def fpp(x, y, u):
    lx = len(x)
    x.reverse()
    y.reverse()
    for qi in range(0, lx):
        x.append([complex(0)])
        y.append([complex(0)])
    x.reverse()
    y.reverse()
    fft(x, u)
    fft(y, u)
    tmp = []
    for elem in range(0, len(x)):
        for el_x in range(0, len(x[elem])):
            for el_y in range(0, len(y[elem])):
                p = x[elem][el_x]
                p *= y[elem][el_y]
                tmp.append(p)
        x[elem] = tmp
        tmp = []
    i_fft(x, u)
    return x


n = int(input())
a = mk_p(n)
ans = []
```

```
inc = []
e_per = 0  # ожидание
h_per = 0  # ожидание половины
for it in range(10):
    b = [[complex(random.randint(1, 25))]]
    d = [[complex(random.randint(1, 25))]]
    c = []
    f = []
    check = []
    for i in range(n):
        check.append(0)
    for i in range(int(n / 2) - 1):
        b.append([complex(random.randint(0, 25))])
        d.append([complex(random.randint(0, 25))])
    for i in range(int(n / 2)):
        c.append(b[i])
        f.append(d[i])
    t = 0
    inc.append(0)
    prev_sum = 0
    for i in range(n):
        j = a[random.randint(0, len(a) - 1)]
        c = []
        f = []
        for k in range(int(n / 2)):
            c.append(b[k])
            f.append(d[k])
        c = fpp(c, f, j)
        for k in range(0, n):
            if len(c[k]) == 1:
                check[k] = 1
        summa = 0
        for k in range(0, n):
            summa += check[k]
        inc[it] += summa - prev_sum
        prev_sum = summa
        if i == 0:
            print("closed percentage 1st step: ", summa / n)
        if summa / n >= 0.5 and t == 0:
            h_per += i
            print("1 half covered steps: ", i, '\n')
            t = 1
        if summa / n >= 0.99:
            e_per += i
            ans.append(i)
            print("steps: ", i, '\n')
            inc[it] /= (i + 1)
            inc[it] /= n
            break
    t = 0
```

```
    if inc[it] > 1:
        inc[it] /= (n * n)
    for i in range(0, n):
        if check[i] == 0:
            if t == 0:
                ans.append(n)
                t = 1
            print(i)
    print("\nSUMMA: ", summa)
    print("\nINC: ", inc[it])

    print('\n')
print("TOTAL:\n")
for it in range(len(ans)):
    print(ans[it], ' ')

print("E: ", e_per / 10, '\n')
print("E(half): ", h_per / 10, '\n')

inc_per = 0
for i_p in range(0, 10):
    print(inc[i_p])
    inc_per += inc[i_p]
print('\n')
print("E(increase): ", inc_per / 10)
```

Для проверки корректности полученных результатов был реализован алгоритм умножения многочленов столбиком:

```
...
def col_multiplication(x, y, s):
    ans = []
    for v in range(0, s):
        ans.append([complex(0)])
    for v in range(0, int(s / 2)):
        for u in range(0, int(s / 2)):
            ans[v + u][0] += x[v][0] * y[u][0]
    return ans
...
```

Проверка показала, что после завершения алгоритма все коэффициенты хранятся в сумме не более, чем двух слагаемых и разница между коэффициентами полученными при применении БПФ и умножении столбиком не превосходит по модулю 10^{-6} для полиномов длины $<= 2^{11}$ и 10^{-3} для полиномов длины $<= 2^{13}$.
Исследование проводилось для полиномов длины $n = 2^k$, где $k \in [7, 12]$. В ходе эксперимента за n запусков рандомизированного алгоритма покрыть 99% не удалось, $\frac{1}{2}$ коэффициентов покрыть также не удалось. Были получены зависимость между длиной полинома и долей покрытия на первом запуске алгоритма и зависимость среднего прироста покрытых коэффициентов от длины полинома. Также было получено среднее число покрытых коэффициентов за n запусков рандомизированного алгоритма. Результаты представим в

табличной форме:

n	E(шагов)	E(1 шаг)	E(1/2)	E(прирост)	E(покрыто)
128	-	0.069141	-	0.002982	48.85
256	-	0.048632	-	0.001626	106.6
512	-	0.039258	-	0.000853	223.5
1024	-	0.026709	-	0.000417	436.75
2048	-	0.016626	-	0.000194	814.7
4096	-	0.006567	-	0.000089	1495.3

Представим полученные результаты в графической форме.

Графическое представление результатов:

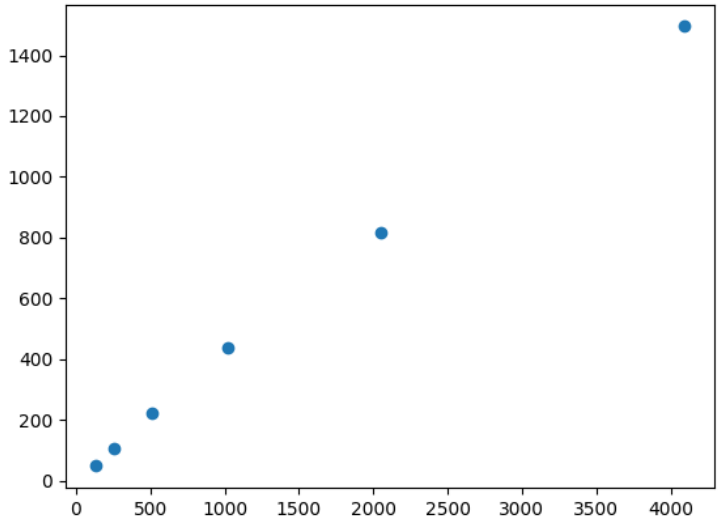

Рис. 1: Зависимость числа покрытых коэффициентов от длины полинома

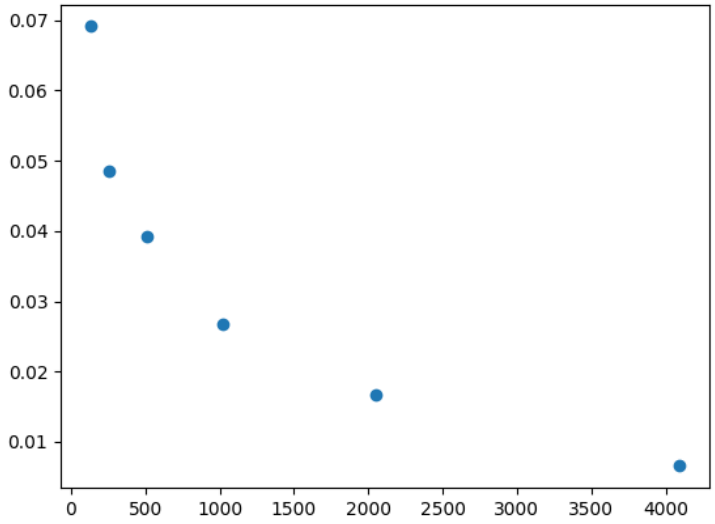

Рис. 2: Зависимость доли покрытия на первом шаге от длины полинома

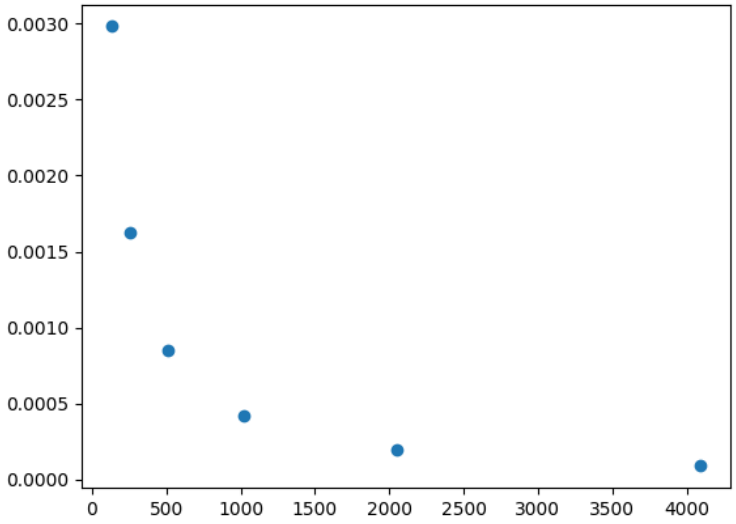

Рис. 3: Зависимость среднего прироста от длины полинома

По полученным результатам можно выдвинуть предположение о линейной зависимости количества покрытых коэффициентов полинома от длины полинома. Для подтверждения гипотезы проведем регрессионный анализ. Воспользуемся пакетом Microsoft EXCEL для построения регрессионной статистики.

ВЫВОД ИТОГОВ

Регрессионная статистика	
Множественный R	0,998548759
R-квадрат	0,997099624
Нормированный R-квадрат	0,99637453
Стандартная ошибка	33,26156868
Наблюдения	6

Дисперсионный анализ

	df	*SS*	*MS*	*F*	*Значимость F*
Регрессия	1	1521352,132	1521352,132	1375,1317	3,15762E-06
Остаток	4	4425,327804	1106,331951		
Итого	5	1525777,46			

	Коэффициенты	*Стандартная ошибка*	*t-статистика*	*P-Значение*	*Нижние 95%*	*Верхние 95%*	*Нижние 95,0%*	*Верхние 95,0%*
Y-пересечение	32,66641791	18,91478361	1,727031014	0,15923461	-19,8494404	85,1822763	-19,84944045	85,18227627
Переменная X 1	0,363306237	0,009797171	37,08276818	3,1576E-06	0,336104928	0,39050755	0,336104928	0,390507545

Рис. 4: Регрессионная статистика

По полученным результатам можно сделать вывод о высокой степени зависимости количества покрытых коэффициентов от длины полинома, о чем свидетельствует высокий коэффициент детерминации равный 0.9970996. Уравнение линейной регрессии имеет вид: $S = 0.36330624 \cdot n + 32.6664179$. Можно сделать вывод о линейной зависимости между количеством покрытых коэффициентов полинома и длиной полинома n.
Зависимость между длиной полинома и долей покрытых коэффициентов на первом шаге и зависимость между длиной полинома и средним приростом покрытых коэффициентов носят обратную зависимость.

5 Выводы

На основе проведенного эксперимента можно сделать следующие выводы:

1. Итоговая ассимптотическая сложность алгоритма равна $O(n\|a\|_1 ln(\|a\|_1) ln(n))$ арифметических операций.

2. Алгоритм, основанный на случайном выборе первообразного корня матрицы Вандермонда, не позволяет получить $\frac{1}{2}$ коэффициентов полинома, кодирующего рюкзак.

3. Наибольшая доля покрытых коэффициентов сосредоточена относительно $\frac{1}{2}$ длины полинома.

Таким образом, полученная модификация алгоритма быстрого произведения многочленов не позволяет принять выдвинутую гипотезу.

6 Метод двойного экспоненциального представления

Определение Пусть $z \in \mathbb{C}$. Двойным экспоненциальным представлением числа z будем называть число $a \in \mathbb{C}$ или пару чисел $a, b \in \mathbb{C}$ при условии, что $|Arg(a) - Arg(b)| < \frac{2\pi}{3}$: $z = e^a$ или $z = e^a + e^b$
В случае $z = e^\alpha, \alpha \in \mathbb{R}$ двойное экспоненциальное представление будем называть каноническим.

Лемма Пусть $y, z, y', z' \in \mathbb{C}$ такие, что $1 + e^y = e^z$ и $1 + e^{y'} = e^{z'}$.
Если $Arg(y) \in \left[-\frac{2\pi}{3}, \frac{2\pi}{3}\right]$ и $|y - y'| \leq \frac{1}{2}$, то тогда существует константа $C > 0$: $|z - z'| \leq C$.

Следствие Пусть $x, y, z, x', y', z' \in \mathbb{C}$ такие, что $e^x + e^y = e^z$ и $e^{x'} + e^{y'} = e^{z'}$. Обозначим $\delta_x = x' - x$, $\delta_y = y' - y$, $\delta_z = z' - z$, и $\varepsilon = max\{|\delta_x|, |\delta_y|\}$. Если $\angle(x, y) < \frac{2\pi}{3}$ и $\varepsilon \leq \frac{1}{4}$, то $\exists C > 0$: $|\delta_z| \leq C \cdot \varepsilon$.

Лемма Пусть $y, y' \in \mathbf{R}_{<0}$ Пусть $y, y' : |y - y'| < \frac{1}{4}|y|$. Пусть $z, z' \in \mathbf{R} : e^z = 1 - e^y, e^{z'} = 1 - e^{y'}$.
Тогда $\exists C > 0 : |z - z'| \leq |y - y'|\,(1 + \frac{C}{|y|})$.
Док-во
$z = \ln(1 - e^y), z' = \ln(1 - e^{y'})$
$|z - z'| = \left|\ln(1 - e^{y'}) - \ln(1 - e^y)\right|$. Пусть $\nu = min\{|y|, |y'|\}$.
Тогда $|z - z'| = \left|\int_\nu^{y'} \frac{e^x}{1-e^x}dx - \int_\nu^y \frac{e^x}{1-e^x}dx\right| = \left|\int_y^{y'} \frac{e^x}{1-e^x}dx\right| = \left|\int_y^{y'} \frac{e^x-1}{1-e^x}dx + \int_y^{y'} \frac{dx}{1-e^x}\right| = \left|y - y' - \int_y^{y'} \frac{dx}{e^x-1}\right|$

$= \left|y - y' - \int_y^{y'} \frac{2dx}{e^{\frac{x}{2}}(\frac{e^{\frac{x}{2}} - e^{-\frac{x}{2}}}{2})}\right| = \left|y - y' - \int_y^{y'} \frac{2dx}{e^{\frac{x}{2}} sh(\frac{x}{2})}\right| = \left|y - y' - \int_y^{y'} \frac{4d\frac{x}{2}}{e^{\frac{x}{2}} sh(\frac{x}{2})}\right|$
Делаем замену $t = \frac{x}{2} \to x = 2t, x \in [y, y'] \to t \in \left[\frac{y}{2}, \frac{y'}{2}\right], t \neq 0$
$-\frac{1}{sh(x)}$ - знакоположительная функция на $\left[\frac{y}{2}, \frac{y'}{2}\right]$ и монотонная, $\frac{1}{e^x}$ - интегрируемая на $\left[\frac{y}{2}, \frac{y'}{2}\right]$, следовательно применима вторая теорема о среднем
$\left|y - y' - \int_y^{y'} \frac{4d\frac{x}{2}}{e^{\frac{x}{2}} sh(\frac{x}{2})}\right| = \left|y - y' + \int_y^{y'} \frac{4d\frac{x}{2}}{e^{\frac{x}{2}}(-sh(\frac{x}{2}))}\right| = \left|y - y' - \frac{4}{sh(\frac{y}{2})}\int_y^\xi \frac{4d\frac{x}{2}}{e^{\frac{x}{2}}}\right|$, где $\xi \in \left[\frac{y}{2}, \frac{y'}{2}\right]$
$\left|y - y' - \frac{4}{sh(\frac{y}{2})}\int_y^\xi \frac{4d\frac{x}{2}}{e^{\frac{x}{2}}}\right| = \left|y - y' - \frac{4}{sh(\frac{y}{2})}e^{-t}\|_{\frac{y}{2}}^{\xi}\right| \leq |y - y'| + \left|\frac{4}{sh(\frac{y}{2})}(e^{-\xi} - e^{-\frac{y}{2}})\right|$
Пусть y мало. Разложим $e^{-\xi}$ и $e^{-\frac{y}{2}}$ в ряд Маклорена.
$|y - y'| + \left|\frac{4}{sh(\frac{y}{2})}(e^{-\xi} - e^{-\frac{y}{2}})\right| = |y - y'| + \left|\frac{4}{sh(\frac{y}{2})}(1 + (-\xi) + \sum_{k=2}^{\infty}\frac{(-1)^k\xi^k}{k!} - 1 + \frac{y}{2} - \sum_{k=2}^{\infty}\frac{(-1)^k(\frac{y}{2})^k}{k!})\right| \leq$
$\frac{|y|}{4} + \left|\frac{4}{sh(\frac{y}{2})}((-\xi) + \sum_{k=2}^{\infty}\frac{(-1)^k\xi^k}{k!} + \frac{y}{2} - \sum_{k=2}^{\infty}\frac{(-1)^k(\frac{y}{2})^k}{k!})\right| \sim \frac{|y|}{4} +$
$+ \left|\frac{8}{y}((-\xi) + \sum_{k=2}^{\infty}\frac{(-1)^k\xi^k}{k!} + \frac{y}{2} - \sum_{k=2}^{\infty}\frac{(-1)^k(\frac{y}{2})^k}{k!})\right| = \frac{|y|}{4} + \left|\frac{8}{y}((-\xi) + \frac{y}{2}) + C\right|$, так как $sh(x) \sim x$ в окрестности 0 и ряды имеют члены порядка ≥ 2. Так как ξ и y имеют порядок y получаем
$\frac{|y|}{4} + \left|\frac{8}{y}((C_1 y) + C\right| \leq CONST$

7 Программная реализация двойного экспоненциального представления

```
#include <iostream>
#include <vector>
#include <istream>
#include <ostream>
#include <complex>
#include <algorithm>
#include <cmath>
#include <random>
using namespace std;
using ll = long long;
using ld = long double;
using cld = complex<long double>;
using vcld = vector<complex<long double>>;
long double pi = 3.1415926535898;
ld tol = 0;
cld zero = { -16 };
ll randmax = 8;
random_device rd;
mt19937 gen(rd());
uniform_int_distribution<> dist(1, 50);

ld ang_comp(cld x, cld y)
{
    if ((x.real() == 0 && x.imag() == 0) || (y.real() == 0 && y.imag() == 0))
        return (ld)0.;
    if (x.real() * y.real() >= tol && x.imag() * y.imag() >= tol)
        return abs(abs(arg(x)) - abs(arg(y)));
    if (x.real() * y.real() <= -tol && x.imag() * y.imag() <= -tol)
        return pi - ang_comp(x, -y);
    if (x.real() * y.real() <= -tol && x.imag() * y.imag() >= tol)
        return abs(arg(x) - arg(y));
    if (x.real() * y.real() >= tol && x.imag() * y.imag() <= -tol)
    {
        if (x.real() >= tol and y.real() >= tol)
            return abs(arg(x)) + abs(arg(y));
        if (x.real() <= -tol && y.real() <= -tol)
            return 2 * pi - abs(arg(x)) - abs(arg(y));
    }
}

class DER
{
    cld a;
    cld b;
public:
    DER();
    DER(cld f, cld l);
```

```
    void print();
    void set_rand();
    DER& operator +=(DER& right);
    DER& operator *= (DER& right);
    DER operator=(DER right);
    DER operator+(DER right);
    DER operator*(DER& right);
    DER& operator/=(ld right);
    DER operator-(DER right);
    friend ostream& operator<<(ostream& stream, const DER& der);
};

using vd = vector<DER>;

DER::DER()
{
    this->a = zero;
    this->b = zero;
}
DER::DER(cld f, cld l = zero)
{
    a = f;
    b = l;
}

void DER::print()
{
    cout << this->a << " " << this->b << endl;
}

void DER::set_rand()
{
    this->a = cld(dist(gen));
    this->b = zero;
}

DER& DER::operator+=(DER& right)
{
    vcld list(4);
    list[0] = exp(this->a);
    list[1] = exp(this->b);
    list[2] = exp(right.a);
    list[3] = exp(right.b);
    while (1)
    {
        if (list.size() == 2)
        {
            if (ang_comp(list[0], list[1]) < 2 * pi / 3)
            {
                list[0] += list[1];
```

```
                list.pop_back();
            }
            break;
        }
        sort(list.begin(), list.end(), [](cld f1, cld f2) { return arg(f1) < arg(f2);
        for (ll i = 0; i < list.size(); i++)
        {
            if (ang_comp(list[i], list[(i + 1) % list.size()]) < 2 * pi / 3)
            {
                list[i] += list[(i + 1) % list.size()];
                list.erase(list.begin() + (i + 1) % list.size());
            }
        }
    }
    this->a = log(list[0]);
    this->b = zero;
    if (list.size() == 2)
        this->b = log(list[1]);
    return *this;
}

DER DER::operator=(DER right)
{
    this->a = right.a;
    this->b = right.b;
    return *this;
}

DER& DER::operator *=(DER& right)
{
    vcld list(4), e_list(4);
    list[0] = this->a + right.a;
    list[1] = this->a + right.b;
    list[2] = this->b + right.a;
    list[3] = this->b + right.b;
    DER ans1, ans2;
    ans1.a = list[0];
    ans1.b = list[1];
    ans2.a = list[2];
    ans2.b = list[3];
    ans1 += ans2;
    *this = ans1;
    return *this;
}

DER DER::operator+(DER right)
{
    DER ans = *this;
    ans += right;
    return ans;
```

```
}

DER DER::operator*(DER& right)
{
    DER ans = *this;
    ans *= right;
    return ans;
}

DER& DER::operator/=(ld right)
{
    cld f = exp(this->a), s = exp(this->b);
    f /= right, s /= right;
    this->a = log(f), this->b = log(s);
    return *this;
}

DER DER::operator-(DER right)
{
    vcld list(4);
    list[0] = exp(this->a);
    list[1] = exp(this->b);
    list[2] = -exp(right.a);
    list[3] = -exp(right.b);
    while (1)
    {
        if (list.size() == 2)
        {
            if (ang_comp(list[0], list[1]) < 2 * pi / 3)
            {
                list[0] += list[1];
                list.pop_back();
            }
            break;
        }
        sort(list.begin(), list.end(), [](cld f1, cld f2) { return arg(f1) < arg(f2);
        for (ll i = 0; i < list.size(); i++)
        {
            if (ang_comp(list[i], list[(i + 1) % list.size()]) < 2 * pi / 3)
            {
                list[i] += list[(i + 1) % list.size()];
                list.erase(list.begin() + (i + 1) % list.size());
            }
        }
    }
    this->a = log(list[0]);
    this->b = zero;
    if (list.size() == 2)
        this->b = log(list[1]);
    return *this;
```

```
}

ostream& operator<<(ostream& stream, const DER& der)
{
    stream << der.a;
    if (der.b != cld(0))
    {
        stream << " + " << der.b;
    }
    return stream;
}

DER root(ll rank, ll pow)
{
    cld j1 = { 0, 1 };
    DER res(exp((2. * pi * pow / rank) * j1));
    return res;
}

vd fft(vd x, ll u, bool invert)
{
    ll n = x.size();
    if (x.size() == 1)
        return x;
    vd ev(x.size() / 2), od(x.size() / 2);
    for (ll i = 0, j = 0; i < x.size(); i += 2, j++)
    {
        ev[j] = x[i];
        od[j] = x[i + 1];
    }
    ev = fft(ev, u, invert);
    od = fft(od, u, invert);
    ld ang = 2. * u * pi / n * (invert ? -1 : 1);
    DER w(0), wn(log(cld{cos(ang), sin(ang)}));
    for (ll i = 0; i < n / 2; i++)
    {
        x[i] = ev[i] + od[i] * w;
        x[n / 2 + i] = (ev[i] - (w * od[i]));
        if (invert)
        {
            x[i] /= 2;
            x[n / 2 + i] /= 2;
        }
        w *= wn;
    }
    return x;
}

vd mult(vd x, vd y, ll u)
{
```

```
    ll n = 1;
    while (n < max(x.size(), y.size())) n <<= 1;
    n <<= 1;
    x.resize(n), y.resize(n);
    x = fft(x, u, 0), y = fft(y, u, 0);
    for (ll i = 0; i < n; i++)
        x[i] *= y[i];
    x = fft(x, u, 1);
    return x;
}

int main()
{
    DER f(cld{ 1, 1 });
    vd a(4), b(4);
    a[0] = DER(1), a[1] = DER(1), a[2] = DER(1), a[3] = DER(1);
    b[0] = DER(1), b[1] = DER(1), b[2] = DER(1), b[3] = DER(1);
    a = mult(a, b, 3);
    for (ll i = 0; i < a.size(); i++)
        cout << a[i] << endl;
}
```

8 Заключение

В работе был рассмотрен и изучен метод применения алгоритма БПФ Кули-Тьюки при поиске оптимума в радаче об ограниченном рюкзаке с целочисленными коэффициентами. Данный метод был проверен на конкретных примерах. Изучены методы аппроксимации комплексных чисел рациональными функциями. Приведена программная реализация модифицированного алгоритма быстрого произведения многочленов для проверки гипотезы о возможности построения рандомизированного алгоритма. Опытным путем установлено, что полученный алгоритм не достигает требуемой сложности и не позволяет вычислить большую часть коэффициентов полинома. Результаты эксперимента подтверждаются построенной регрессионной статистикой. Для решения задачи о (min, +)-свертке был рассмотрен метод двойного экспоненциального представления, были улучшены результаты оценки. Построена программная реализация (min, +)-свертки с двойным экспоненциальным представлением на языке C++.

9 Ссылки и использованная литература

[1] Yurii Nesterov - Fast Fourier Transform and its Applications to Integer Knapsack Problems - 07.06.2004
[2] E. B. Saff, A. Shönhage, R. S. Varga - Geometric Convergence to e^{-z} by Rational Functions with Real Poles - 18.07.1975
[3] Kalle Leppälä, Tapani Matala-aho, Topi Törmä - Rational approximations of the exponential function at rational points - 11.06.2018